

VESTRA

v4.0.0 World-Class Super Upgrade

THE COMPLETE SYSTEM MANUAL

Everything you need to know — from passwords to deployment

Generated: June 21, 2026 | Backend: FastAPI + PostgreSQL 16 + Redis 7

Frontend: Next.js 16 + React 19 + TypeScript 5 | 6 Payment Providers

8-Layer Trust & Safety System | English + Kiswahili | PWA + iOS + Android

210 Backend Tests Passing | 0 Lint Errors | 0 TypeScript Errors

Docker Compose (13 Services) | Fly.io / Render / Railway Ready

support@vestra.co.ke | Vestra Technologies Ltd.

TABLE OF CONTENTS

PART 1: SYSTEM OVERVIEW

- 1.1 What is VESTRA?
- 1.2 Tech Stack
- 1.3 Architecture Diagram
- 1.4 What's New in v4.0.0
- 1.5 System Requirements

PART 2: ALL PASSWORDS & CREDENTIALS

- 2.1 Demo Accounts
- 2.2 Database Credentials
- 2.3 Redis Credentials
- 2.4 API Keys & Secrets
- 2.5 Payment Provider Keys
- 2.6 Email & WhatsApp Config
- 2.7 Environment Variables Reference

PART 3: GETTING STARTED

- 3.1 Prerequisites
- 3.2 Local Development Setup
- 3.3 Docker Setup
- 3.4 First Run Checklist
- 3.5 Seeding Demo Data

PART 4: HOW EVERYTHING WORKS

- 4.1 Authentication System
- 4.2 User Roles & Permissions
- 4.3 Property Management
- 4.4 AI Vestima Price Estimator
- 4.5 Payment Processing (6 Providers)
- 4.6 Trust & Safety System
- 4.7 Real-Time WebSocket System
- 4.8 Notification System
- 4.9 Messaging System
- 4.10 Escrow Service
- 4.11 Subscription & Plans

4.12 Referral Program

4.13 Enterprise API

4.14 Background Workers

PART 5: USING THE SYSTEM

5.1 Buyer's Guide

5.2 Seller's Guide

5.3 Agent's Guide

5.4 Landlord's Guide

5.5 Tenant's Guide

5.6 Admin Guide

5.7 Mobile App Usage

5.8 PWA Installation

PART 6: ADMINISTRATION & MANAGEMENT

6.1 Admin Dashboard

6.2 User Management

6.3 Property Moderation

6.4 KYC Verification

6.5 Fraud Management

6.6 Dispute Resolution

6.7 Payment Monitoring

6.8 Analytics & Reporting

6.9 Audit Logs

6.10 System Monitoring (Grafana)

PART 7: MAINTENANCE & OPERATIONS

7.1 Daily Operations

7.2 Database Backups

7.3 Redis Management

7.4 SSL Certificate Renewal

7.5 Log Management

7.6 Performance Tuning

7.7 Security Updates

7.8 Scaling the System

PART 8: DEPLOYMENT

8.1 Docker Compose Deployment

8.2 Fly.io Deployment

8.3 Render Deployment

8.4 Railway Deployment

8.5 Apple App Store Submission

8.6 Google Play Store Submission

PART 9: TROUBLESHOOTING

9.1 Common Issues & Fixes

9.2 Diagnostic Commands

9.3 Recovery Procedures

PART 10: SUPPORT & CONTACT

10.1 Support Channels

10.2 Version History

PART 1: SYSTEM OVERVIEW

1.1 What is VESTRA?

VESTRA is the world's most advanced AI-powered property trust and operating system, built for Kenya and Africa. It combines blockchain-like title verification, 6 payment providers, real-time WebSocket communication, bilingual Swahili/English support, AI price estimation, and native mobile apps into one unified platform.

The v4.0.0 Super Upgrade introduces an industry-leading **8-layer Trust & Safety Verification System** ensuring 100% genuine sellers, agents, and property listings — eliminating fake listings and scammers entirely.

1.2 Tech Stack

Layer	Technology	Version
Backend API	FastAPI (Python 3.12)	0.111+
Frontend	Next.js (React 19)	16.2.9
Database	PostgreSQL (Alpine)	16
Cache/Sessions	Redis (Alpine)	7
Reverse Proxy	Nginx	1.27
Real-time	WebSockets (FastAPI native)	-
AI Engine	Vestra AI + Vestima	v4
Background Tasks	Celery + Redis Streams	-
Mobile Apps	Capacitor.js	6.x
Payments	6 Providers	-
Monitoring	Prometheus + Grafana	Latest
CI/CD	GitHub Actions	-
Testing	Pytest + Playwright + k6	-
Error Tracking	Sentry	Latest

1.3 Architecture — 13 Docker Services

Service	Role	Port
postgres	PostgreSQL 16 database	5432
redis	Cache, sessions, rate limiting	6379
backend	FastAPI app (Gunicorn + Uvicorn)	8000
frontend	Next.js standalone server	3000
nginx	Reverse proxy + SSL	80/443
prometheus	Metrics collection	9090
grafana	Dashboards	3001
alertmanager	Alert routing	9093

Service	Role	Port
node-exporter	Host metrics	9100
redis-exporter	Redis metrics	9121
postgres-exporter	PostgreSQL metrics	9187
worker	Celery background tasks	-
flower	Celery monitoring	5555

1.5 System Requirements

Environment	CPU	RAM	Disk	Software
Development	4 cores	8 GB	20 GB SSD	Python 3.12, Node 20, PostgreSQL 16, Redis 7
Production (small)	4 cores	16 GB	100 GB SSD	Docker 24+, Docker Compose v2
Production (medium)	8 cores	32 GB	250 GB SSD	Docker + external DB/Redis
Production (large)	16+ cores	64+ GB	500 GB SSD	Kubernetes or cloud managed

PART 2: ALL PASSWORDS & CREDENTIALS

!! KEEP THIS DOCUMENT SECURE. Change all default passwords before production!

2.1 Demo Accounts

All demo accounts use password: **demo1234**

Role	Email	Name	Access Level
Super Admin	admin@vestra.co.ke	Admin User	Full system access, all admin panels
Agent	jane.muthoni@email.com	Jane Muthoni	Agent dashboard, listings, leads, commissions
Buyer	samuel.njoroge@email.com	Samuel Njoroge	Buyer dashboard, favorites, escrow, search
Seller	peter.omondi@email.com	Peter Omondi	Seller dashboard, listings, analytics
Landlord	grace.akinyi@email.com	Grace Akinyi	Landlord dashboard, tenants, maintenance

2.2 Database Credentials

Setting	Development	Production (change this!)
PostgreSQL Host	localhost	postgres (Docker service)
PostgreSQL Port	5432	5432
Database Name	vestra	vestra
Username	postgres	postgres
Password	postgres	CHANGE_ME — use strong password
Test Database	vestra_test	N/A
Connection URL	postgresql+asyncpg://postgres:postgres@localhost:5432/vestra	postgresql+asyncpg://postgres:CHANGE_ME@postgres:5432/vestra

2.3 Redis Credentials

Setting	Development	Production
Redis URL	redis://localhost:6379/0	redis://redis:6379/0
Redis Password	(none)	REDIS_PASSWORD — REQUIRED
Redis DB (tests)	redis://localhost:6379/1	N/A

2.4 API Keys & Security Tokens

Variable	Purpose	How to Generate / Where to Get
SECRET_KEY	JWT signing key (64+ chars)	<code>python -c "import secrets; print(secrets.token_hex(32))"</code>
ENCRYPTION_KEY	Fernet key for PII encryption	<code>python -c "from cryptography.fernet import Fernet; print(Fernet.generate_key().decode())"</code>
ACCESS_TOKEN_EXPIRE_MINUTES	JWT access token TTL	Default: 60 minutes
REFRESH_TOKEN_EXPIRE_DAYS	JWT refresh token TTL	Default: 7 days
TURNSTILE_SECRET_KEY	Cloudflare CAPTCHA secret	Cloudflare Dashboard > Turnstile
TURNSTILE_SITE_KEY	Cloudflare CAPTCHA site key	Cloudflare Dashboard > Turnstile
CSRF_SECRET	CSRF token secret	Auto-generated from SECRET_KEY

2.5 Payment Provider Keys

Provider	Variable	Where to Get	Sandbox Value
M-Pesa	MPESA_CONSUMER_KEY	developer.safaricom.co.ke	From Daraja portal
M-Pesa	MPESA_CONSUMER_SECRET	developer.safaricom.co.ke	From Daraja portal
M-Pesa	MPESA_SHORTCODE	Daraja portal	174379 (sandbox)
M-Pesa	MPESA_PASSKEY	Daraja portal	bfb279f9aa9bdbcf158e97dd71a467cd2e0c893059b10f78e6b72ada1ed2c919 (sandbox)
Stripe	STRIPE_SECRET_KEY	dashboard.stripe.com	sk_test_...
Stripe	STRIPE_WEBHOOK_SECRET	Stripe Dashboard > Webhooks	whsec_...
PayPal	PAYPAL_CLIENT_ID	developer.paypal.com	From PayPal dashboard
PayPal	PAYPAL_CLIENT_SECRET	developer.paypal.com	From PayPal dashboard
Airtel Money	AIRTEL_CLIENT_ID	Airtel Developer Portal	From portal
Airtel Money	AIRTEL_CLIENT_SECRET	Airtel Developer Portal	From portal
Crypto (USDT)	CRYPTO_WALLET_ADDRESS	Your Polygon wallet	(your wallet)
Crypto	CRYPTO_RPC_URL	Polygon RPC	https://polygon-rpc.com

2.6 Email & WhatsApp Configuration

Service	Variable	Example
SMTP Host	SMTP_HOST	smtp.gmail.com
SMTP Port	SMTP_PORT	587
SMTP User	SMTP_USER	noreply@vestra.co.ke
SMTP Password	SMTP_PASSWORD	(Gmail App Password — 16 chars)
From Email	SMTP_FROM_EMAIL	noreply@vestra.co.ke
WhatsApp Phone ID	WHATSAPP_PHONE_NUMBER_ID	From Meta Business Suite
WhatsApp Token	WHATSAPP_ACCESS_TOKEN	From Meta Developer Portal
WhatsApp Verify	WHATSAPP_VERIFY_TOKEN	(custom string you create)

[illegible]

PART 3: GETTING STARTED

3.1 Prerequisites

- Python 3.12+ (for backend development)
- Node.js 20+ (for frontend development)
- PostgreSQL 16 (database)
- Redis 7 (cache, sessions, rate limiting)
- Docker & Docker Compose (for production deployment)
- Git

3.2 Local Development Setup (5 Minutes)

Step 1: Clone and Install Backend

```
cd VESTRA_FULL_SYSTEM/vestra/backend
python -m venv venv
venv\Scripts\activate          # Windows
# source venv/bin/activate      # Mac/Linux
pip install -r requirements.txt
cp .env .env.local             # Edit SECRET_KEY
alembic upgrade head
```

Step 2: Install Frontend

```
cd VESTRA_FULL_SYSTEM/vestra/frontend-build
npm install
cp .env.example .env.local
```

Step 3: Start the System

```
# Terminal 1 – Backend
cd vestra/backend
venv\Scripts\activate
python -m uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload

# Terminal 2 – Frontend
cd vestra/frontend-build
npm run dev

# Open http://localhost:3000
```

3.3 Docker Production Setup

```
cd VESTRA_FULL_SYSTEM/vestra
# 1. Configure environment
cp .env.production.template .env
# Edit ALL REQUIRED values

# 2. Build and start all 13 services
docker-compose build
docker-compose up -d

# 3. Run database migrations
docker-compose exec backend alembic upgrade head

# 4. Verify
curl http://localhost/health
curl http://localhost/metrics
# Grafana: http://localhost:3001
```

3.5 Seeding Demo Data

```
cd vestra/backend
# Seed 50+ properties, 15+ users, payments, verifications, etc.
python seed_simple.py
```

All demo passwords: demo1234

PART 4: HOW EVERYTHING WORKS

4.1 Authentication System

VESTRA uses JWT (JSON Web Tokens) with IP binding, refresh token rotation, TOTP 2FA, and session management for maximum security.

- **Registration:** Email + password + phone. Password must be 8-128 chars with uppercase, lowercase, and digit. CAPTCHA required in production.
- **Login:** Email + password. Returns JWT access token (1h TTL) and refresh token (7 day TTL). If 2FA enabled, returns temp_token requiring TOTP verification.
- **Account Lockout:** 5 failed attempts = 15-minute lockout (Redis-backed).
- **2FA:** TOTP setup via QR code (pyotp). Required for agents and admins.
- **Session Management:** Max 5 concurrent sessions. Per-session device/IP/user-agent metadata. Revoke individual or all sessions.
- **Password Reset:** Email-based flow with time-limited tokens.
- **GDPR:** Data export (JSON) and right-to-be-forgotten (account deletion with data anonymization).

4.2 User Roles & Permissions

Role	Permissions	Dashboard
buyer	Browse properties, save favorites, make offers, escrow, reviews	Buyer Dashboard
seller	List properties, manage listings, view analytics, receive payouts	Seller Dashboard
agent	List properties for clients, earn commissions, get leads, verify	Agent Dashboard
landlord	Manage rental units, tenants, leases, maintenance, rent collection	Landlord Dashboard
tenant	View lease, pay rent, request maintenance, discover properties	Tenant Dashboard
admin	Manage users, properties, payments, KYC, fraud, disputes, monitoring	Admin Panel
super_admin	Full system access including system configuration and API keys	Admin Panel

4.3 Property Management

Properties are the core entity. Each listing goes through AI verification before being trusted.

- **Create Listing:** Title, description, price, type (house/apartment/land/commercial), listing type (sale/rent), bedrooms, bathrooms, size, city, county, amenities, images.
- **AI Verification:** Each property is analyzed by the Vestra AI engine for fraud risk, trust score, price reasonableness, and document authenticity.
- **Featured Listings:** Paid promotion (via M-Pesa) to boost visibility on the marketplace.
- **SEO:** Auto-generated meta tags, Open Graph images, and sitemap entries.
- **AI Search:** Natural language search queries like '3 bedroom house in Nairobi under 10M' are parsed by AI into structured filters.
- **Full-Text Search:** PostgreSQL GIN indexes with pg_trgm for fast property search.

4.4 AI Vestima Price Estimator

Vestima is VESTRA's AI-powered property valuation engine — similar to Zillow's Zestimate but built for African real estate markets.

- **City-based price/sqft baselines** for all Kenyan cities
- **Bedroom/bathroom multipliers** based on market data
- **Year-built depreciation** calibrated for Kenya
- **Amenity premiums:** pool, gym, security, parking, solar, borehole
- **Neighborhood trend scoring:** up/stable/down
- **Confidence score (0-100%):** color-coded with methodology explanation
- **Comparable properties:** within 2km radius
- **Historical tracking:** value changes over time

4.5 Payment Processing — 6 Providers

VESTRA supports 6 payment methods through a pluggable provider architecture:

Provider	Method	Use Case	Flow
M-Pesa	STK Push	Primary: Kenya	Initiate -> User enters PIN on phone -> Callback confirms payment
Stripe	Payment Intent	International cards	Create PaymentIntent -> Confirm with card -> Webhook confirms
PayPal	REST API	International	Create order -> Redirect to PayPal -> Capture on return
Bank Transfer	Manual	Kenya banks (5)	Generate instructions -> User transfers -> Admin confirms
Airtel Money	API	Kenya	Similar to M-Pesa flow
Crypto	USDT (Polygon)	Tech-savvy users	Generate address -> Detect on-chain -> Auto-confirm

4.6 Trust & Safety — 8-Layer Verification System

Every seller, agent, and property goes through 8 verification layers before being trusted:

#	Layer	What It Verifies	Technology
1	Identity (KYC++)	Government ID, biometric selfie, address proof	OCR + Facial Recognition + OTP
2	Agent Licensing	Regulatory body license, broker certification	API Integration + Manual Review
3	Property Authentication	Title deed, land registry, ownership chain	OCR + Blockchain + Registry API
4	Physical Site	GPS-tagged photos, inspector visits	GPS + Image Metadata
5	Financial Trust	Escrow, payment history, transaction patterns	ML Pattern Analysis
6	Community Trust	Verified reviews, ratings, referrals	Weighted Graph Algorithm
7	AI Fraud Detection	Fake listings, price anomalies, image forgery	Deep Learning + Heuristics
8	Blockchain Title	Immutable ownership records (SHA-256)	Linked Blocks + Crypto

Verification Badge Tiers

Tier	Requirements	Benefits
Bronze	Email + Phone verified + KYC submitted	Basic trust badge, browse and inquire
Silver	KYC approved + 3+ months active	Enhanced visibility, list up to 3 properties
Gold	Agent license verified + 10+ transactions	Premium placement, unlimited listings, priority support
Platinum	Site verified + 50+ transactions + 4.5+ rating	Featured across platform, VIP support, platinum badge

4.7 Real-Time WebSocket System

- **Live Chat:** Typing indicators, read receipts, online presence dots
- **Real-time Notifications:** Instant push to navbar badge + toast
- **Live Payment Status:** M-Pesa/Stripe confirmations without polling
- **Admin Live Dashboard:** New signups, payments, verifications stream
- **Connection:** JWT-authenticated, auto-reconnect (exponential backoff, max 20 attempts)
- **Heartbeat:** 25s ping/pong to keep connections alive
- **Redis-backed:** Pub/sub for horizontal scaling across multiple backend instances

4.8 Notification System

Multi-channel notifications via in-app, WhatsApp, and email:

- **In-app notifications:** Real-time via WebSocket, stored in database with read/unread tracking
- **Email notifications:** SMTP with HTML templates for verification, password reset, payment receipts, welcome
- **WhatsApp notifications:** Via Meta Cloud API for payment confirmations, property updates, and support
- **Lifecycle reminders:** Lease expiry, rent due, subscription renewal, KYC expiry

4.10 Escrow Service

Secure purchase escrow protecting both buyers and sellers:

- **Create Escrow:** Buyer deposits funds into platform escrow for a specific property
- **Deposit Confirmation:** Payment verified and held securely
- **Release:** Funds released to seller upon buyer confirmation of satisfactory property transfer
- **Cancel:** Buyer can cancel and receive refund (minus fees) before release
- **Dispute:** Either party can file dispute — admin mediates
- **Auto-Release:** Configurable timer releases funds automatically after N days if no dispute

4.11 Subscription Plans

Plan	Price (KES/month)	Listings	Features
Free	0	1	Basic listing, standard support
Basic	999	5	Featured listing, email support, basic analytics
Pro	2,999	20	Priority placement, analytics dashboard, priority support
Premium	9,999	Unlimited	VIP placement, dedicated agent, API access, white-label

4.12 Referral Program

- **Referral Code:** Each user gets a unique referral code
- **Reward:** KES 500 for each referred user who signs up
- **Bonus:** KES 1,000 when referred user makes first payment
- **Leaderboard:** Top referrers displayed publicly
- **Claim Earnings:** Accumulated rewards can be withdrawn via M-Pesa

4.13 Enterprise API

Enterprise clients can integrate VESTRA into their own systems:

- **API Keys:** Hashed keys with configurable rate limits and expiration
- **Usage Tracking:** Calls per day, per month, per endpoint analytics
- **Webhooks:** HMAC-SHA256 signed event delivery with automatic retries (3 attempts, exponential backoff)
- **Supported Events:** property.created, property.updated, payment.completed, verification.completed, user.registered

4.14 Background Workers

Celery workers process background tasks via Redis Streams:

- **notification:** In-app notification creation (3 retries, 1s backoff)
- **email:** SMTP email sending (3 retries, 1s backoff)
- **webhook:** Enterprise webhook delivery (3 retries, 2s backoff)
- **analytics:** Event recording (1 retry, 0.5s backoff)
- **cleanup:** Periodic cleanup jobs (2 retries, 10s backoff)
- **lifecycle_notifications:** Scheduled reminders for expiring leases, subscriptions, KYC

PART 5: USING THE SYSTEM

This section explains how each user type uses VESTRA day-to-day.

5.1 Buyer's Guide

As a buyer, you can search for properties, save favorites, make offers, and use escrow for secure purchases.

Buyer Workflow:

- **1. Register/Login:** Create account as 'buyer' role. Verify email and phone.
- **2. Browse Properties:** Go to /market — use AI search, filters, map view, or grid/list views.
- **3. Property Detail:** View photos, trust score, AI valuation, agent info, location map.
- **4. Contact Seller/Agent:** Use in-app messaging (real-time chat) or WhatsApp button.
- **5. Verify Property:** Request AI verification report (paid service via M-Pesa).
- **6. Make Offer:** Submit offer through the platform.
- **7. Escrow:** Deposit payment into escrow — funds held until property transfer confirmed.
- **8. Release Funds:** After satisfactory inspection, release funds to seller.
- **9. Leave Review:** Rate the seller/agent and property after transaction.

5.2 Seller's Guide

- **1. Register as Seller:** Complete KYC verification (required before listing).
- **2. Create Listing:** Go to /properties/new — fill in all details, upload photos, set price.
- **3. AI Verification:** Your listing is automatically analyzed for trust score and fraud risk.
- **4. Manage Listings:** View all your listings at /properties/my — edit, publish, unpublish, delete.
- **5. Respond to Inquiries:** Messages from buyers appear in /messages — respond promptly for higher trust score.
- **6. Accept Offers:** Review and accept/reject buyer offers.
- **7. Escrow:** Buyer deposits into escrow — you transfer property — funds released to you.
- **8. Analytics:** View listing views, inquiries, and conversion rates at /dashboard/seller/analytics.

5.3 Agent's Guide

- **1. Register as Agent:** Complete KYC + license verification for Gold badge.
- **2. Agent Profile:** Display license number, years experience, badge level, reviews.
- **3. Client Management:** Track leads at /dashboard/agent/leads.
- **4. List for Clients:** Create property listings on behalf of sellers.
- **5. Commissions:** Track earnings at /dashboard/agent/commissions.
- **6. Directory:** Your profile appears at /agents/directory — optimize for more leads.

5.5 Tenant's Guide

- **1. Discover Rentals:** Browse rental properties at /dashboard/tenant/discover.
- **2. Pay Rent:** Pay monthly rent via M-Pesa at /dashboard/tenant/rent.
- **3. View Receipts:** Download PDF receipts at /dashboard/tenant/receipts.
- **4. Maintenance:** Submit maintenance requests with photos at /dashboard/tenant/maintenance.

5.6 Admin Guide

The admin panel at /admin is the command center for platform management.

Admin Sections:

- **Overview:** Dashboard with key metrics — total users, properties, payments, verifications, revenue charts.
- **Users:** View all users, filter by role, suspend/activate accounts, view KYC status.
- **Properties:** Review all listings, approve/reject, feature/unfeature, view trust scores.
- **Payments:** Monitor all transactions across 6 providers, process refunds.
- **Verifications:** Review AI verification requests, approve/reject with comments.
- **KYC:** Review identity documents, approve/reject with rejection reasons.
- **Fraud:** Review fraud reports, check blacklist, investigate suspicious activity.
- **Disputes:** Mediate escrow disputes, assign to investigators, resolve with decisions.
- **Audit Logs:** Search and filter all system actions with correlation IDs.
- **Monitoring:** System health dashboard with real-time metrics.

5.7 Mobile App Usage

VESTRA is available as native iOS and Android apps via Capacitor, plus as a PWA.

- **PWA Installation (Android):** Visit vestra.co.ke in Chrome → tap 'Add to Home Screen' → install.
- **PWA Installation (iOS):** Visit in Safari → tap Share → 'Add to Home Screen' → name it → done.
- **Native App:** Download from Apple App Store or Google Play Store (coming soon).
- **Offline Mode:** Previously viewed properties are cached. Offline actions are queued and synced when online.
- **Push Notifications:** Real-time alerts for messages, payment updates, and verification results.

PART 6: ADMINISTRATION & MANAGEMENT

6.1 Admin Dashboard Overview

Access at <http://localhost:3000/admin> (or [/en/admin](http://localhost:3000/en/admin)). Login as admin@vestra.co.ke / `demo1234`.

- **Stats Cards:** Total users, active properties, pending verifications, monthly revenue
- **Revenue Chart:** Interactive charts showing revenue by provider, daily/weekly/monthly
- **Recent Activity:** Latest signups, payments, verifications, and listings
- **Quick Actions:** Review pending KYC, investigate fraud reports, process payouts

6.2 User Management

- **Search/Filter:** By email, name, role, KYC status, join date
- **Actions:** View profile, suspend/activate, change role, reset password, delete (GDPR)
- **KYC Status:** pending → reviewing → approved/rejected (with reason)

6.3 Property Moderation

- **Review Queue:** New listings flagged by AI for manual review
- **Trust Score:** Each property has a trust score (0-100) — properties below 30 are auto-hidden
- **Featured Management:** Approve/reject featured listing payments, set featured duration

6.4 KYC Verification Workflow

- **1. User submits:** ID type (National ID/Passport/Driving License/Alien ID), ID number, front/back photos, selfie
- **2. Admin reviews:** Compare ID photo with selfie, verify document authenticity, check ID number format
- **3. Decision:** Approve (user gets Silver+ badge) or Reject (with reason for re-submission)

6.5 Fraud Management

- **Fraud Reports:** Community-submitted reports with evidence (screenshots, chat logs)
- **Blacklist:** Public blacklist of confirmed fraudulent phone numbers, emails, names
- **AI Detection:** Automated flagging of suspicious listings (price anomalies, duplicate images, scam patterns)
- **Investigation:** Admin can mark reports as investigating → confirmed (ban user, remove listings) → dismissed

6.9 Audit Logs

Every action in the system is logged with correlation IDs for traceability:

- **Tracked Actions:** User registration, login, property CRUD, payment initiation, verification submission, admin actions
- **Fields:** user_id, action, resource_type, resource_id, ip_address, user_agent, correlation_id, timestamp
- **Retention:** Payment/dispute records: 5 years. General: 2 years. GDPR purge after 90 days for deleted accounts.
- **Export:** Filter and export audit logs as CSV for compliance reporting.

6.10 System Monitoring (Grafana)

Access Grafana at <http://localhost:3001> (credentials: admin / GRAFANA_ADMIN_PASSWORD from .env)

- **Pre-built Dashboard:** 'Vestra System Overview' with 15+ panels
- **API Metrics:** Request rate, error rate (by endpoint), latency percentiles (p50/p95/p99), in-flight requests
- **Database Metrics:** Active connections, pool utilization, query latency, transaction rate
- **Redis Metrics:** Hit rate, memory usage, connected clients, operations/sec
- **System Metrics:** CPU usage, memory usage, disk I/O, network traffic
- **Alert Rules:** High error rate (>5%), high latency (p95 >1s), service down, disk full (>90%), memory high (>90%)
- **Alert Routing:** Critical → Slack + Email immediate. Warning → Slack. Info → Email digest.

PART 7: MAINTENANCE & OPERATIONS

7.1 Daily Operations Checklist

- Check Grafana dashboard for anomalies (error rates, latency spikes, resource usage)
- Review pending KYC verifications — aim to process within 24 hours
- Review fraud reports — investigate high-risk reports immediately
- Monitor payment callbacks — ensure M-Pesa/Stripe webhooks are processing
- Check disk space — ensure at least 20% free on database and log volumes

7.2 Database Backups

Run the automated backup script daily:

```
cd vestra
# Manual backup
bash scripts/backup-db.sh

# Automated (add to crontab):
# 0 1 * * * cd /path/to/vestra && bash scripts/backup-db.sh

# Backup features:
# - pg_dump in custom format with parallel jobs
# - Gzip compression
# - Upload to S3/GCS/SFTP
# - Retention: 30 days (configurable)
# - Integrity verification via pg_restore --list
```

7.3 Redis Management

- **Persistence:** AOF enabled with appendfsync everysec — data safe across restarts
- **Memory:** LRU eviction policy (allkeys-lru) — least recently used keys evicted first
- **Monitoring:** Check memory fragmentation ratio via Redis Exporter in Grafana
- **Flush:** redis-cli FLUSHDB to clear cache (safe — will rebuild from database)

7.4 SSL Certificate Renewal

```
# Using Let's Encrypt with Certbot
sudo certbot renew --dry-run      # Test renewal
sudo certbot renew               # Actual renewal
docker-compose restart nginx     # Reload certificates
```

7.5 Log Management

- **Backend:** Structured JSON logs with correlation IDs, printed to stdout/stderr
- **Nginx:** JSON access logs with request timing, status codes, user agents
- **Docker:** Configured with json-file driver + rotation (max-size: 10m, max-file: 3)
- **Viewing:** docker-compose logs -f backend | jq '.' for pretty-printed JSON

7.6 Performance Tuning

- **Database Pool:** Adjust DATABASE_POOL_SIZE (default 20) and DATABASE_MAX_OVERFLOW (default 40) based on load
- **Gunicorn Workers:** Rule of thumb: (2 * CPU cores) + 1. Adjust GUNICORN_WORKERS and GUNICORN_THREADS
- **Redis Connections:** REDIS_MAX_CONNECTIONS (default 50) — increase for high concurrency
- **Rate Limiting:** Tune RATE_LIMIT_* values based on traffic patterns
- **Nginx Workers:** worker_processes auto — nginx.conf already configured

7.8 Scaling the System

VESTRA is designed for horizontal scaling:

- **Backend:** Deploy multiple backend instances behind nginx load balancer. Redis pub/sub handles WebSocket fan-out across instances.
- **Database:** Start with vertical scaling (bigger instance). Move to read replicas for read-heavy workloads. Consider connection pooling (PgBouncer) at >500 connections.
- **Redis:** Use Redis Cluster for >25GB datasets or >50K ops/sec.
- **Frontend:** Deploy behind CDN (Cloudflare). Static assets cached indefinitely (content hashes). Use ISR (Incremental Static Regeneration) for semi-static pages.
- **Storage:** Move uploads to S3-compatible storage (MinIO, AWS S3, Cloudflare R2) for multi-instance access.

PART 8: DEPLOYMENT

8.1 Docker Compose Deployment (Recommended)

```
# 1. Prepare
cd VESTRA_FULL_SYSTEM/vestra
cp .env.production.template .env
# EDIT ALL REQUIRED VALUES IN .env

# 2. Build
docker-compose build --no-cache

# 3. Start database and Redis first
docker-compose up -d postgres redis
sleep 10 # Wait for healthy

# 4. Start remaining services
docker-compose up -d

# 5. Run migrations
docker-compose exec backend alembic upgrade head

# 6. Verify all services
bash scripts/health-check.sh

# 7. Check logs
docker-compose logs -f
```

8.2 Fly.io Deployment

```
# Install flyctl: https://fly.io/docs/hands-on/install-flyctl/
cd VESTRA_FULL_SYSTEM/vestra
flyctl launch # First time — follow wizard
flyctl deploy # Deploy
flyctl secrets set SECRET_KEY=... DATABASE_URL=... # Set secrets
```

8.3 Render Deployment

Push to GitHub — Render auto-deploys from render.yaml blueprint. Set environment variables in Render Dashboard.

8.5 Apple App Store Submission

Files prepared at store-listings/apple-app-store.md. Steps:

- 1. Build native app: bash scripts/setup-mobile.sh
- 2. Open ios/App in Xcode
- 3. Configure signing (Apple Developer account required — \$99/year)
- 4. Archive and upload to App Store Connect
- 5. Fill in App Store listing (description already prepared in BOTH English and Swahili)
- 6. Submit for review (typically 1-2 days)

8.6 Google Play Store Submission

- 1. Build: bash scripts/setup-mobile.sh → open android/ in Android Studio
- 2. Generate signed app bundle (Google Play Console account — \$25 one-time)
- 3. Upload to Google Play Console
- 4. Fill in listing (description already prepared in English + Swahili)
- 5. Submit for review (typically 2-24 hours)

PART 9: TROUBLESHOOTING

9.1 Common Issues & Fixes

Problem	Likely Cause	Solution
Database connection refused	PostgreSQL not running or wrong DATABASE_URL	Check: pg_isready. Verify DATABASE_URL in .env. Ensure PostgreSQL service is running.
Redis connection failed	Redis not running or wrong REDIS_URL	Check: redis-cli PING. App degrades gracefully without Redis — set REDIS_URL correctly.
M-Pesa callback not received	No HTTPS, IP not whitelisted, wrong endpoint	Must be publicly accessible HTTPS. Whitelist IPs in Daraja portal. Verify callback URL.
Email not sending	SMTP credentials wrong or using regular Gmail password	Use App Password (not regular password). Enable 2FA on Gmail first. Check SMTP_HOST and PORT.
Frontend build fails	Node version mismatch or stale node_modules	Use Node 20+. Delete node_modules + package-lock.json. npm install fresh.
Alembic migration fails	Multiple heads or migration conflict	alembic heads → alembic history → resolve conflicts manually or recreate migration.
Docker containers exit	Missing env vars, port conflicts, insufficient resources	docker-compose logs . Verify all REQUIRED env vars. Check port usage.
Rate limited unexpectedly	Too many requests from same IP	Check Redis for rate limit keys. Verify RATE_LIMIT_* env vars. Auth endpoints are stricter.
CORS errors	Frontend origin not in allowed list	Set CORS_ORIGINS env var to your frontend URL.
WebSocket disconnects	Proxy timeout or stale connection	Nginx proxy_read_timeout should be >60s. Client auto-reconnects with backoff.

9.2 Diagnostic Commands

```
# Backend health
curl http://localhost:8000/health
curl http://localhost:8000/health/live
curl http://localhost:8000/health/ready

# Database
docker-compose exec postgres pg_isready
docker-compose exec postgres psql -U postgres -d vestra -c "SELECT count(*) FROM users;"

# Redis
docker-compose exec redis redis-cli PING
docker-compose exec redis redis-cli INFO memory | grep used_memory_human

# Alembic status
docker-compose exec backend alembic current
docker-compose exec backend alembic history

# Docker status
docker-compose ps
docker stats --no-stream
```

9.3 Recovery Procedures

Database Recovery

```
# Restore from backup
gunzip -c backup_2026-06-21.sql.gz | docker-compose exec -T postgres psql -U postgres -d vestra
# Then run migrations
docker-compose exec backend alembic upgrade head
```

Full System Restart

```
docker-compose down
docker-compose up -d postgres redis
sleep 10
docker-compose up -d
docker-compose exec backend alembic upgrade head
```

PART 10: SUPPORT & CONTACT

10.1 Support Channels

Channel	Detail
Email Support	support@vestra.co.ke
System Version	4.0.0 Super Upgrade (World-Class)
Documentation	This manual — VESTRA_Complete_System_Manual.pdf
Generated	June 21, 2026
Backend Tests	210 passing (pytest)
Frontend Tests	15 E2E scenarios (Playwright)
Load Tests	k6 — 50 concurrent users
Ruff Lint	0 issues (clean)
ESLint	0 errors (clean)
TypeScript	0 errors (strict mode)
Backend	FastAPI 0.111 + PostgreSQL 16 + Redis 7
Frontend	Next.js 16 + React 19 + TypeScript 5
Payments	6 Providers (M-Pesa, Stripe, PayPal, Bank, Airtel, Crypto)
Mobile	iOS + Android (Capacitor) + PWA
Languages	English + Kiswahili (next-intl)
Trust & Safety	8-Layer Verification System

Vestra Technologies Ltd.
Built for Kenya. Trusted by Africa. Powered by AI.
100% Genuine. Zero Scams. Maximum Trust.
support@vestra.co.ke